

ASP.NET Core in Action

Chapter 19: Building custom MVC and Razor Pages components

19 February 2024

Q1. Which one allows you to customize behavior of existing HTML makeup?

- A. View component
- B. Tag Helpers
- C. Html helpers
- D. View templates

Answer: B

[Page 641]

Q2. What does the input Tag Helper use to set the <input> tag's name and id properties?

- A. value of the property
- B. name of the property
- C. value of the name of the [Display] annotation attribute
- D. None of the above

Answer: B

[Page 642]

```
<input type="text" asp-for="FirstName" />
```

Will produce

```
<input type="text" asp-for="FirstName" id="FirstName" name="FirstName"/>
```

Q3. What should you do to create a custom tag helper?

- A. Create a class from Inheriting from TagHelper
- B. Create a class from Inheriting from TagHelper and override process or process the Process() or ProcessAsync() method
- C. Create a class from Inheriting from TagHelper and write the Process() or ProcessAsync() method
- D. Create a class from Inheriting from HtmlElement and override Render() or RenderAsync() method

Answer: B

[Page 643]

Q4. You created a tag helper class CompanyInfoTagHelper, how would you use it in razor page or view template?

- A. <CompanyInfoTagHelper />
- B. <company-info />
- C. <company-info-tag />
- D. <company-info-tag-helper />

Answer: B

[Page 644]

Q5. You created MyTagHelper in MyAssembly, which one does not correctly register the helper?

- A. @addTagHelper MyAssembly.MyTagHelper, MyAssembly
- B. @addTagHelper *, .MyAssembly
- C. @addTagHelper MyAssembly.MyTagHelper
- D. None

Answer: C

[Page 646]

Q6. Where do you register your custom tag helper?

- A. in the _ViewStart.cshtml file

-
- B. in the _ViewImports.cshtml file
 - C. in the _Layout.cshtml file
 - D. in the Program.cs file

Answer: B

[Page 646]

Q7. What should you do to create a custom middleware to use the core application middleware pipeline?

- A. Create a class inheriting from RequestDelegate and override the Invoke method?
- B. Create a class with constructor with DI injection of RequestDelegate object and write the Invoke method
- C. Create a class with constructor with DI injection of HttpRequest object and write the Invoke method
- D. Create a class with constructor with DI injection of HttpContext object and write the Invoke method

Answer: B

[Page 619]

Custom middleware components don't have to derive from a specific base class or implement an interface. Middleware classes should have a constructor that takes a RequestDelegate object and have an Invoke function with a signature similar to

```
public Task Invoke(HttpContext context);
```

Q8. You created a custom middleware MyMiddleware. Which code should you write to use it in the middleware pipeline?

- A. `app.UseMiddleware( context => new MyMiddleware(context));`
- B. `app.UseMiddleware<MyMiddleware>();`
- C. `app.UseMiddleware( (context, next)=> new MyMiddleware(context, next));`
- D. `app.UseMiddleware( async (context, next)=> new MyMiddleware(context, next));`

Answer: B

[Page 620]

Q10. Which code adds a custom middleware with creating a class?

- A.

```
app.Use(async (context, next) =>
{
    /* ...*/
})
```
- B.

```
app.Use(async (context, next) =>
{
    /* ...*/
    await next(context)
})
```
- C.

```
app.Use(async (context, next) =>
{
    /* ...*/
    await context(next)
})
```
- D.

```
app.Use(async (context, next) =>
{
    await next(context)
    /* ...*/
})
```

Answer: B

[Page 620]